

Data Structures – Unit 05

Bucharest University of Economic Studies

Stacks and Queues



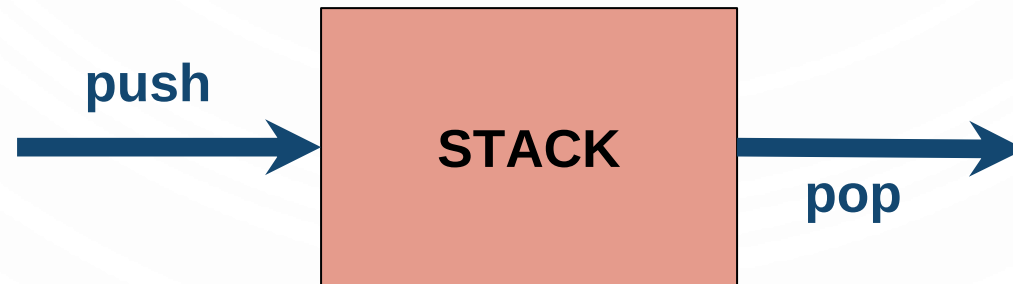
Agenda

- Definition
- Graphical representation
- Internal interpretation
- Characteristics
- Operations
- Implementations



Stack

It is a linear structure in which the two basic operations, insertion and deletion are made on a LIFO based rule. These operations are made in a single part of the structure called the head of the stack, or the top.



Stack

- A logical structure implemented with different type of data structures: linked lists or arrays;
- A homogenous structure with elements of the same type;
- Basic operations:
 - Adding an element, called PUSH operation
 - Removing an element, called POP operation
 - Looking at the top element, called PEEK operation

```
struct Stack{  
    int info;  
    Stack * next;  
};
```



Stack as an ADT

The implementation rule is
LIFO – Last In First Out

The pop operation from a NULL stack
is considered an error

The push operation into a stack is
considered a limitation

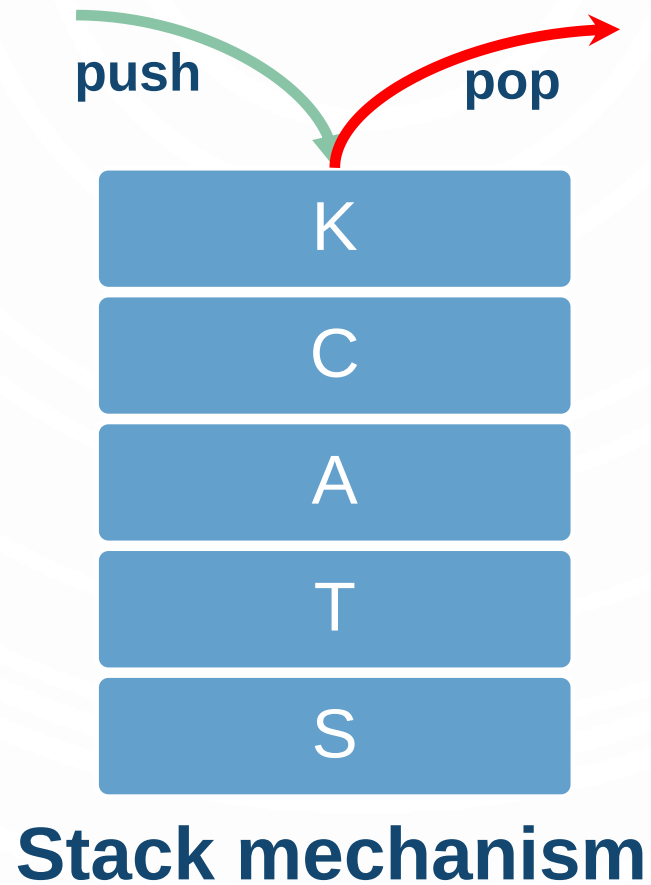
The element from the top of a stack is
the only accessible element



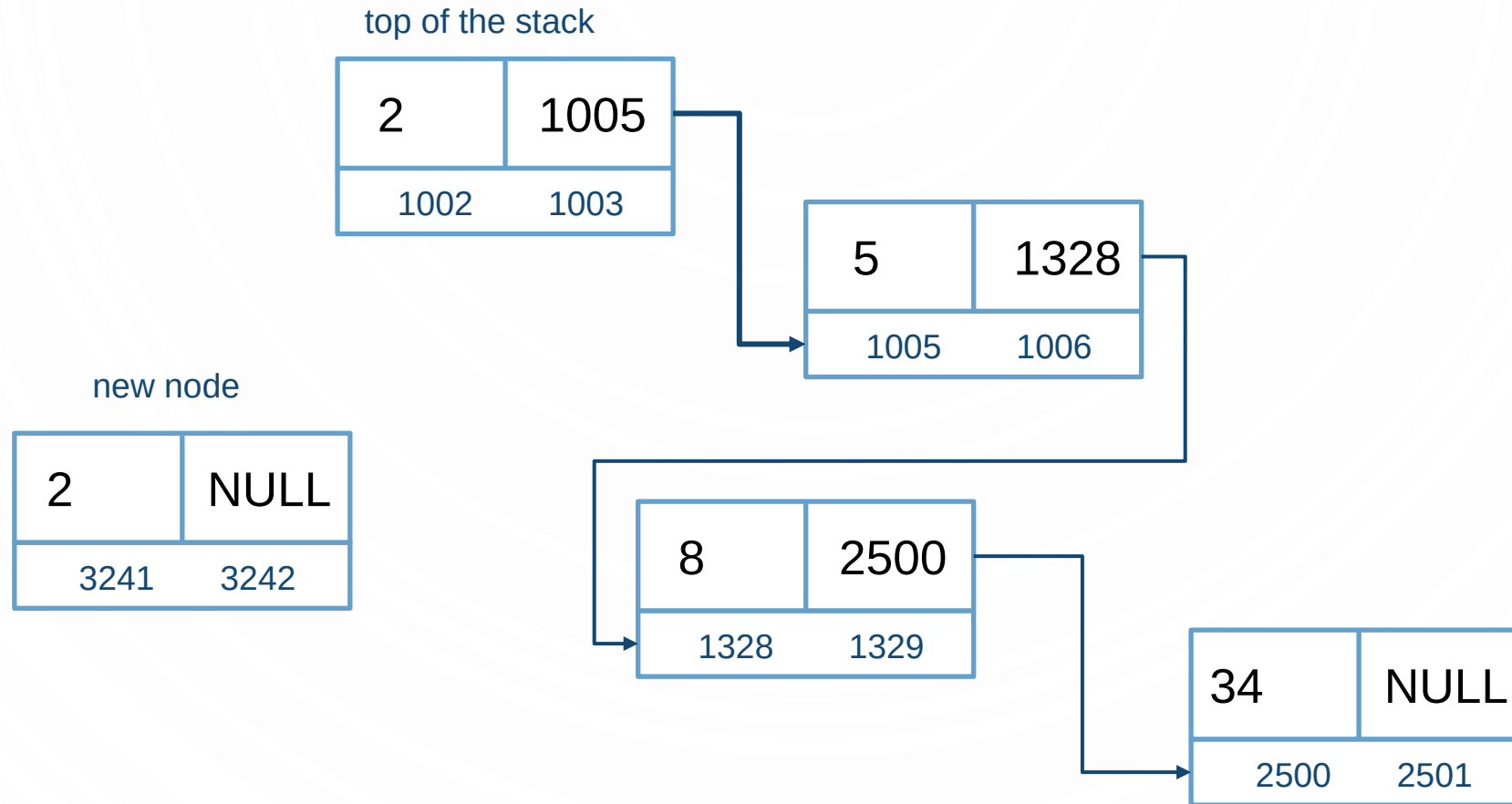
2
4
1
3
6



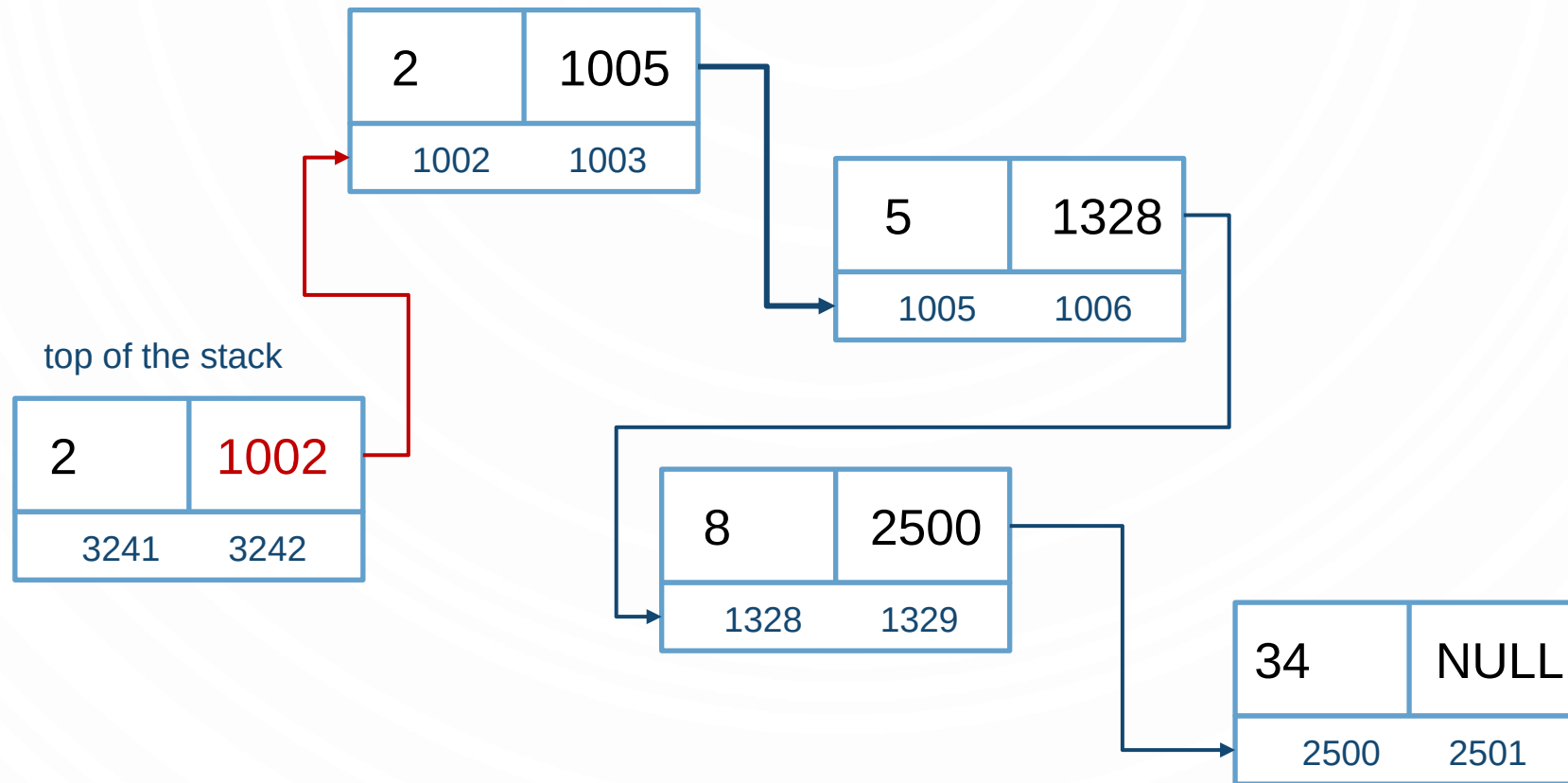
Stack – graphical representation



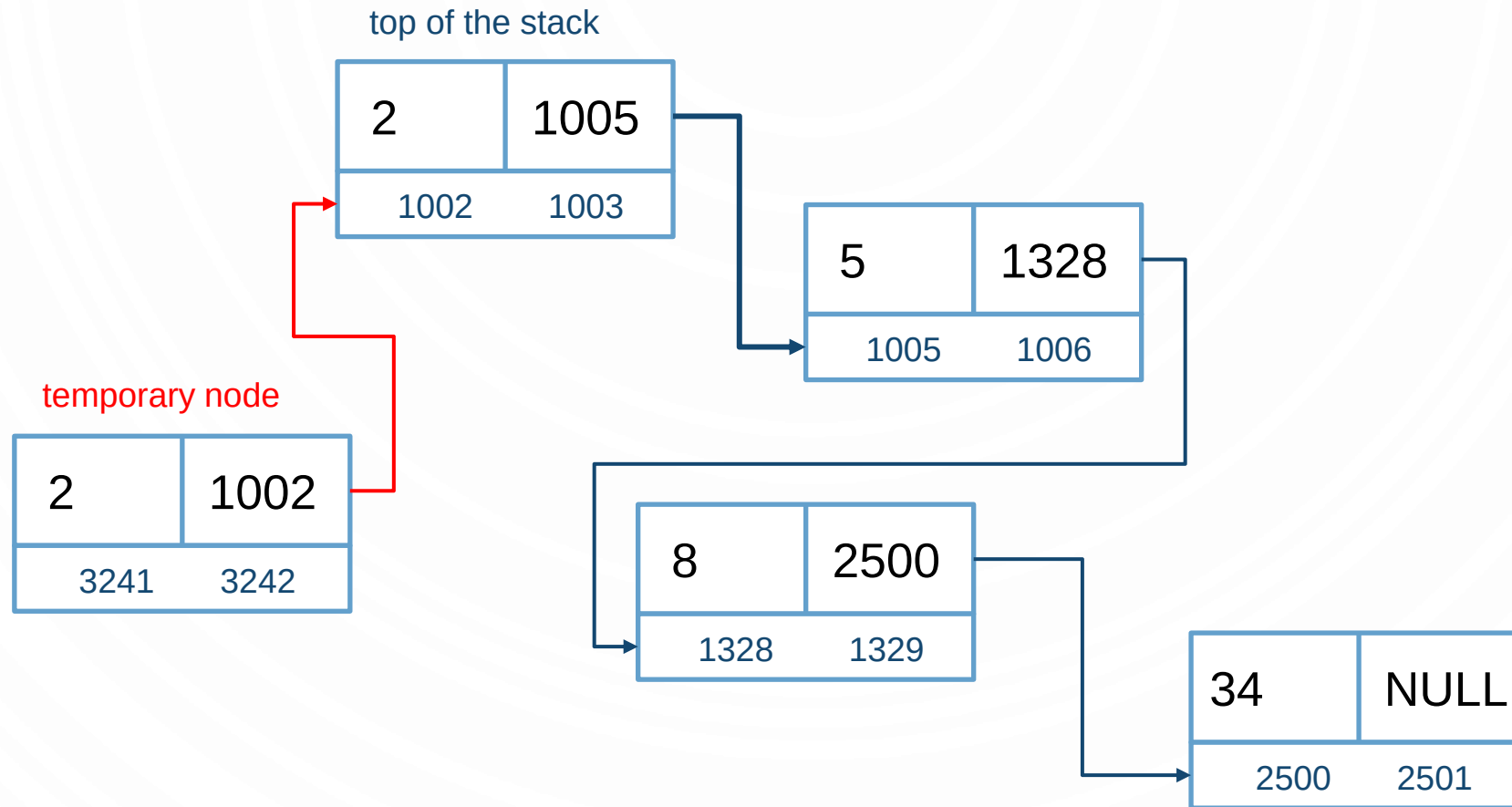
Stacks with Linked Lists



Stacks with Linked Lists



Stacks with Linked Lists



Function call simulation

```
int suma(int x, int y) {  
    int z = 0, r = 2;  
    z = x + y + r;  
    return z;  
}
```

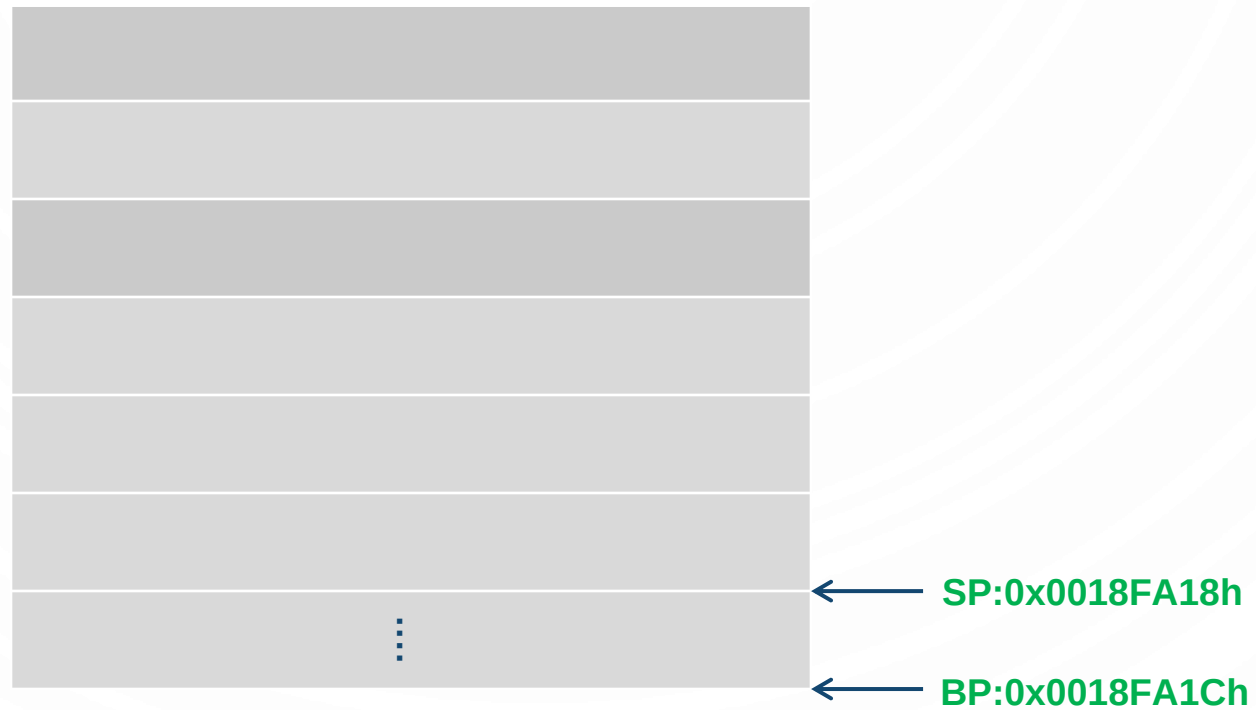
```
void main(){  
    int a = suma(3,8);  
}
```

0x00f11464h



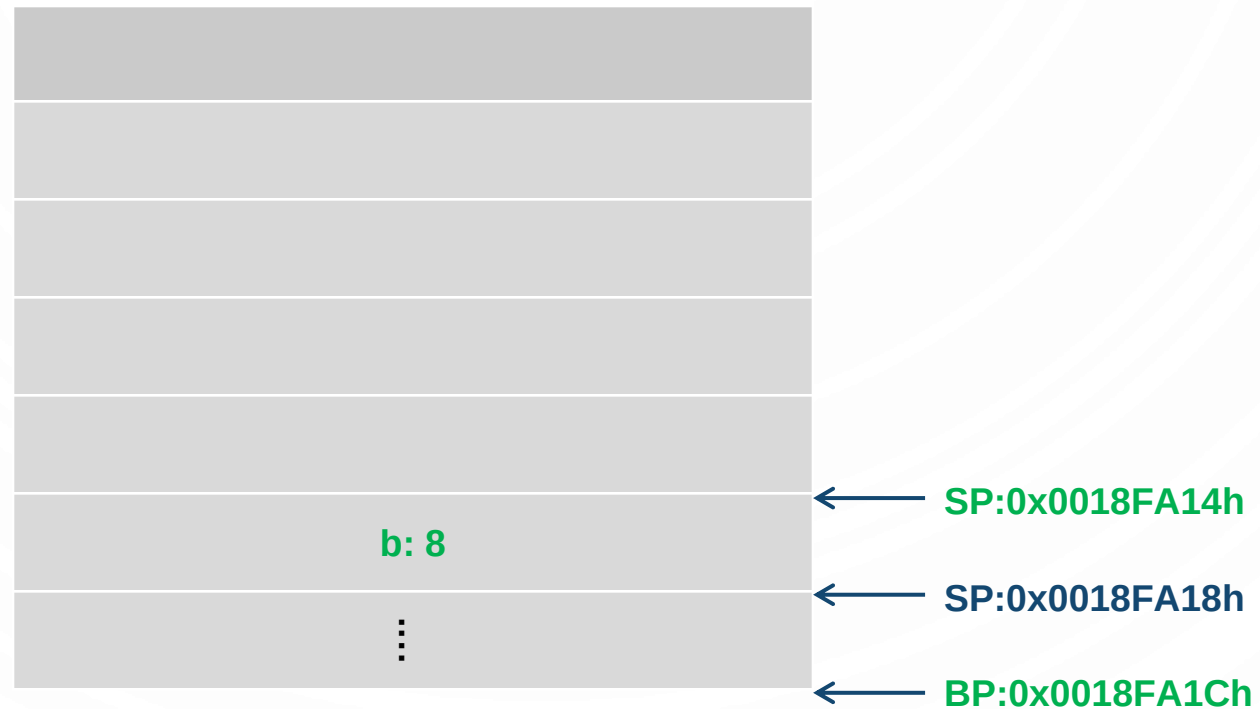
Function call simulation

initialization

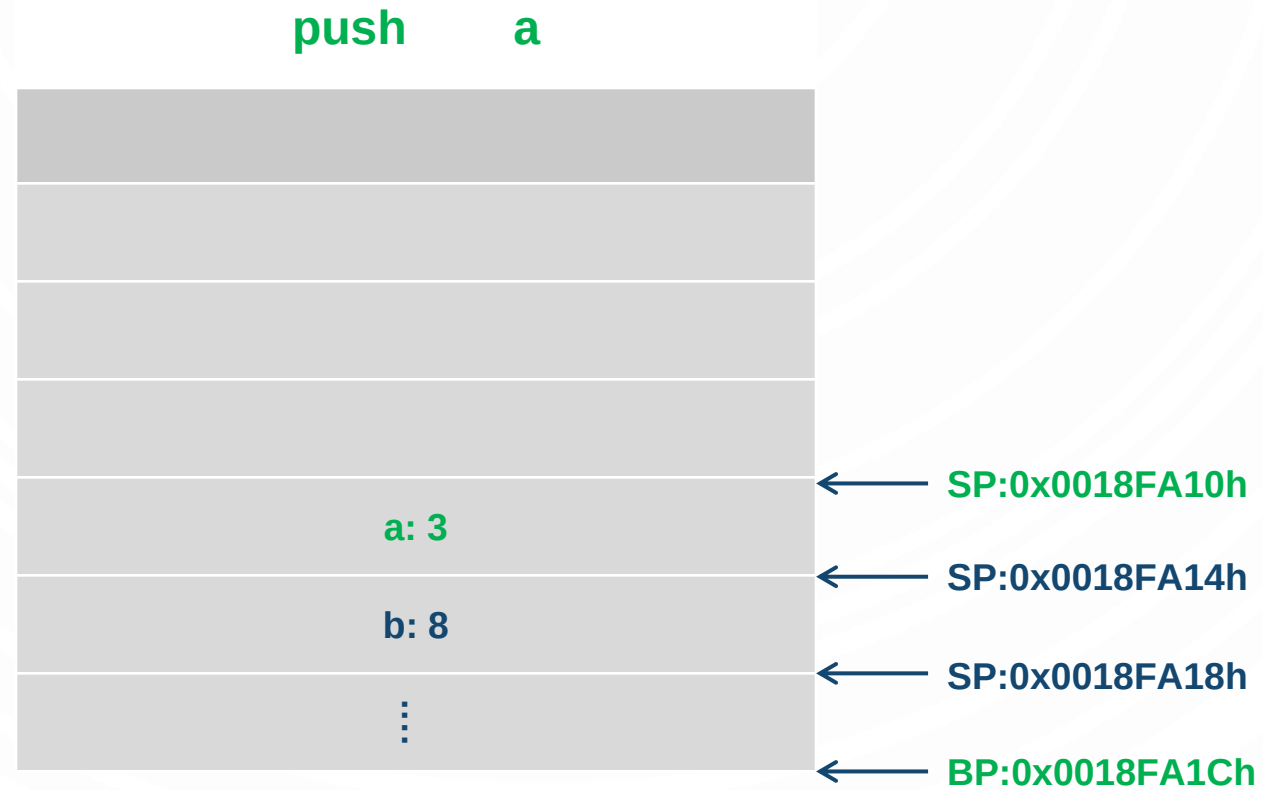


Function call simulation

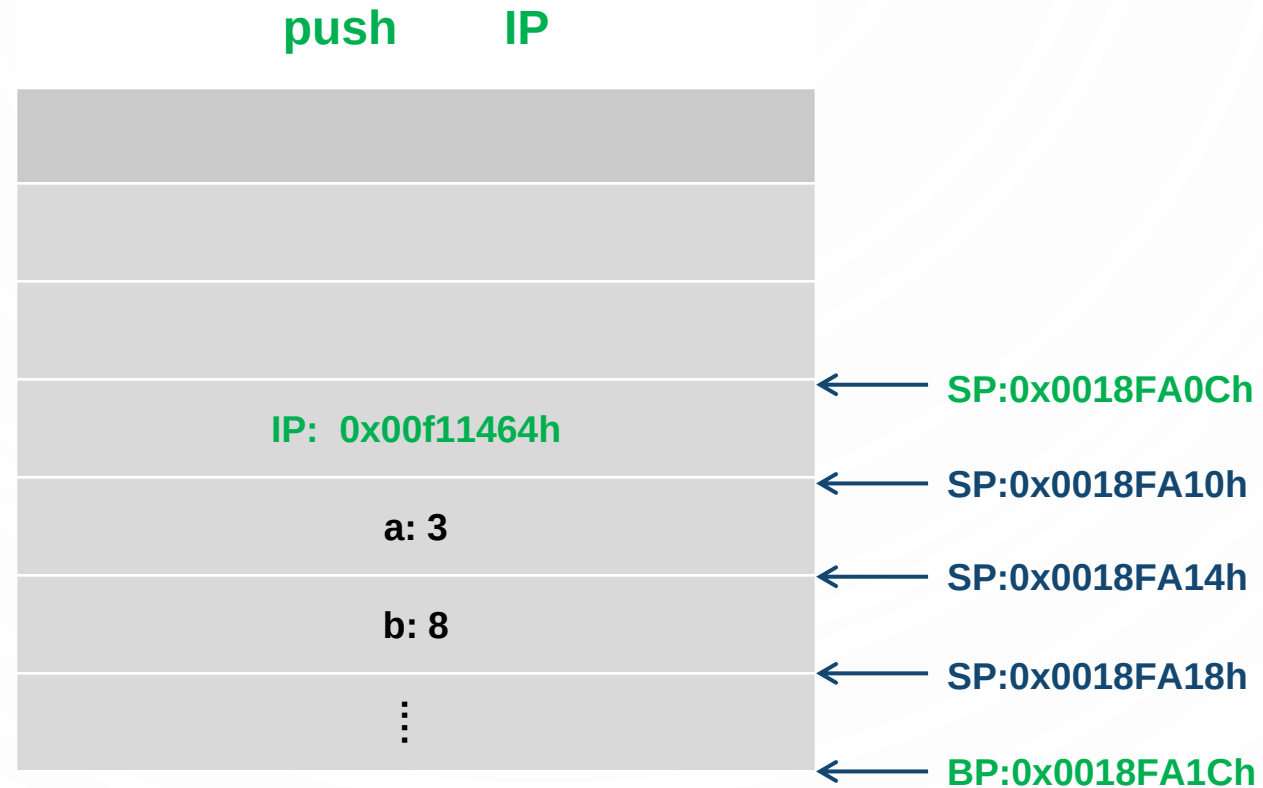
push



Function call simulation

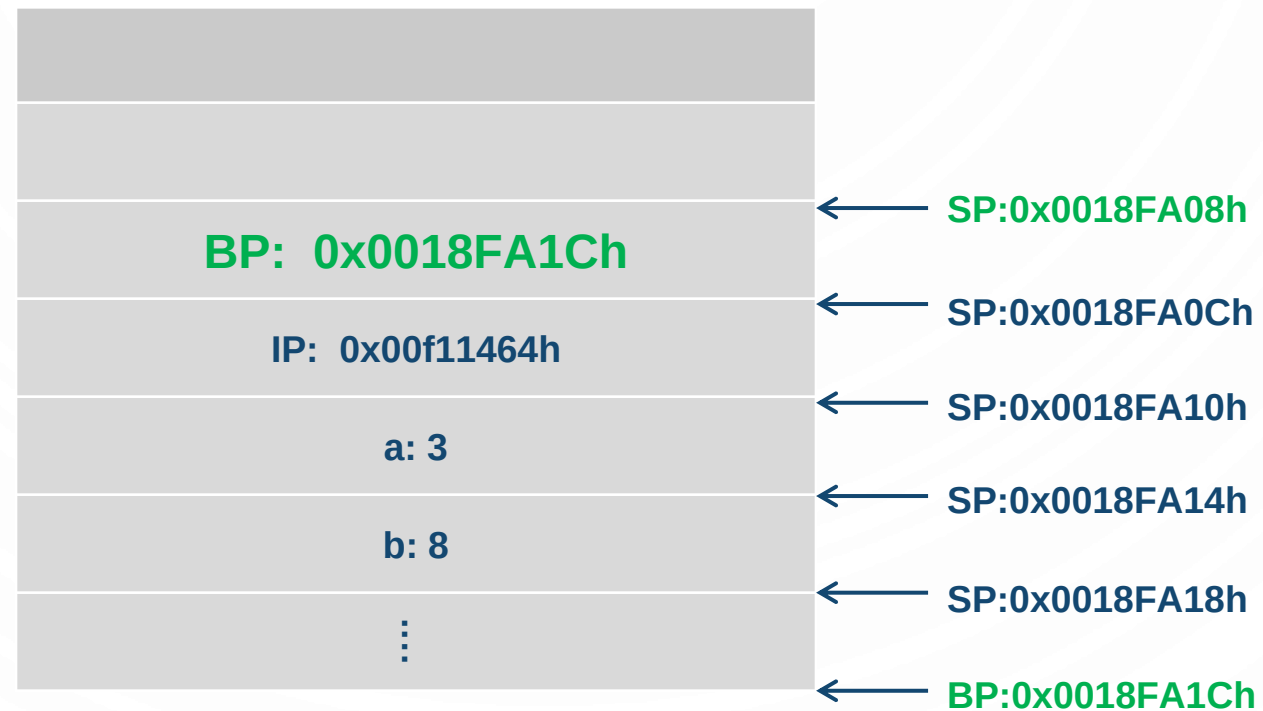


Function call simulation

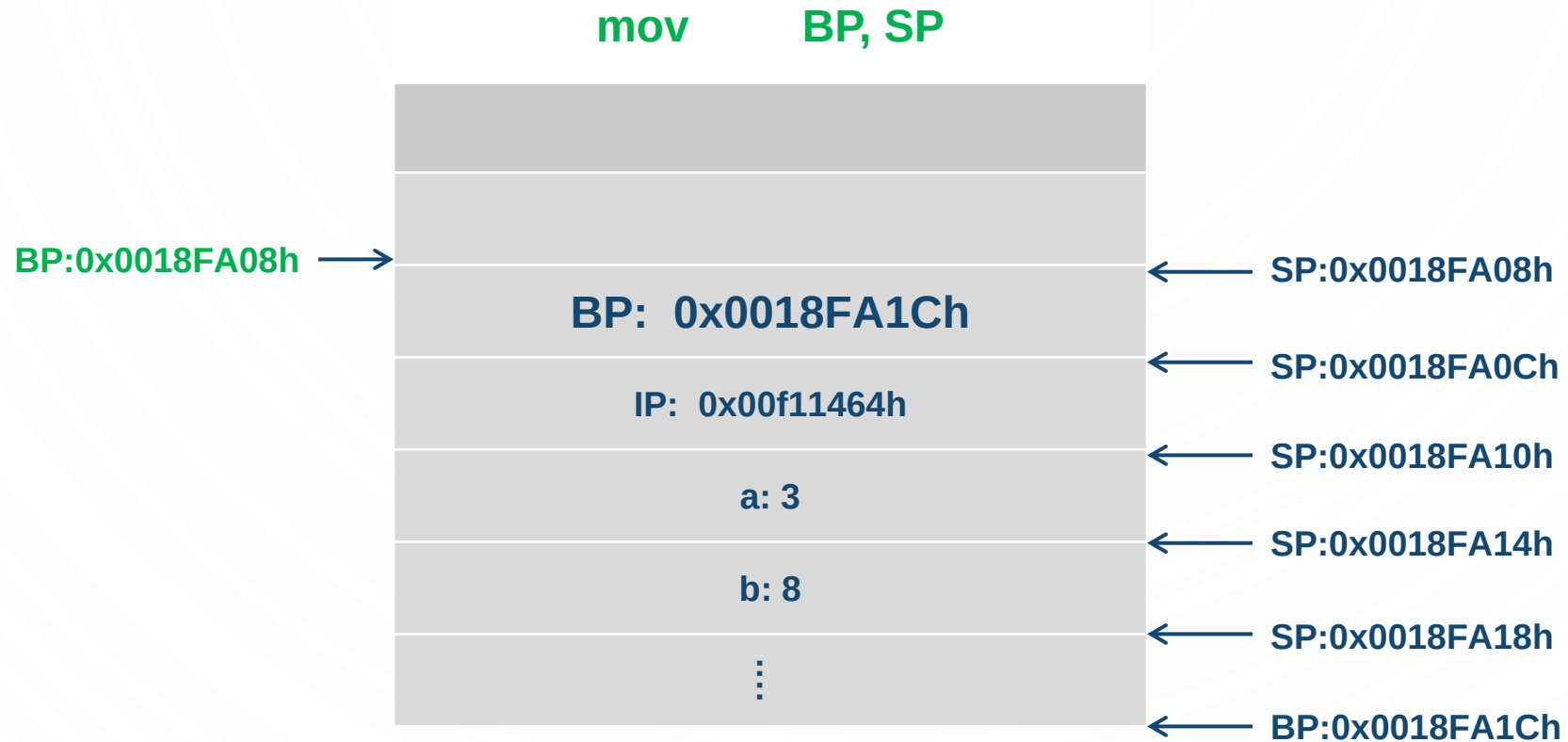


Function call simulation

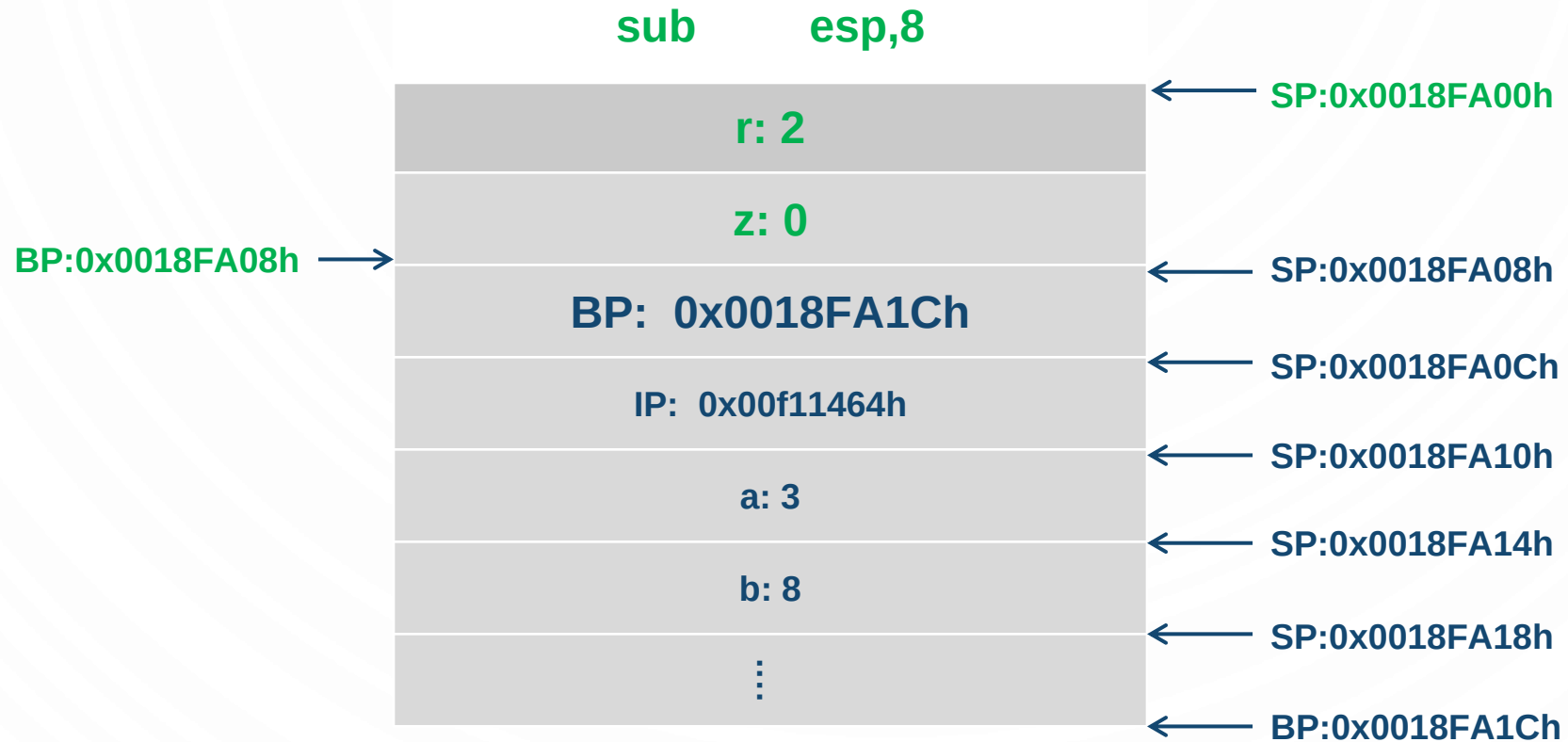
push BP



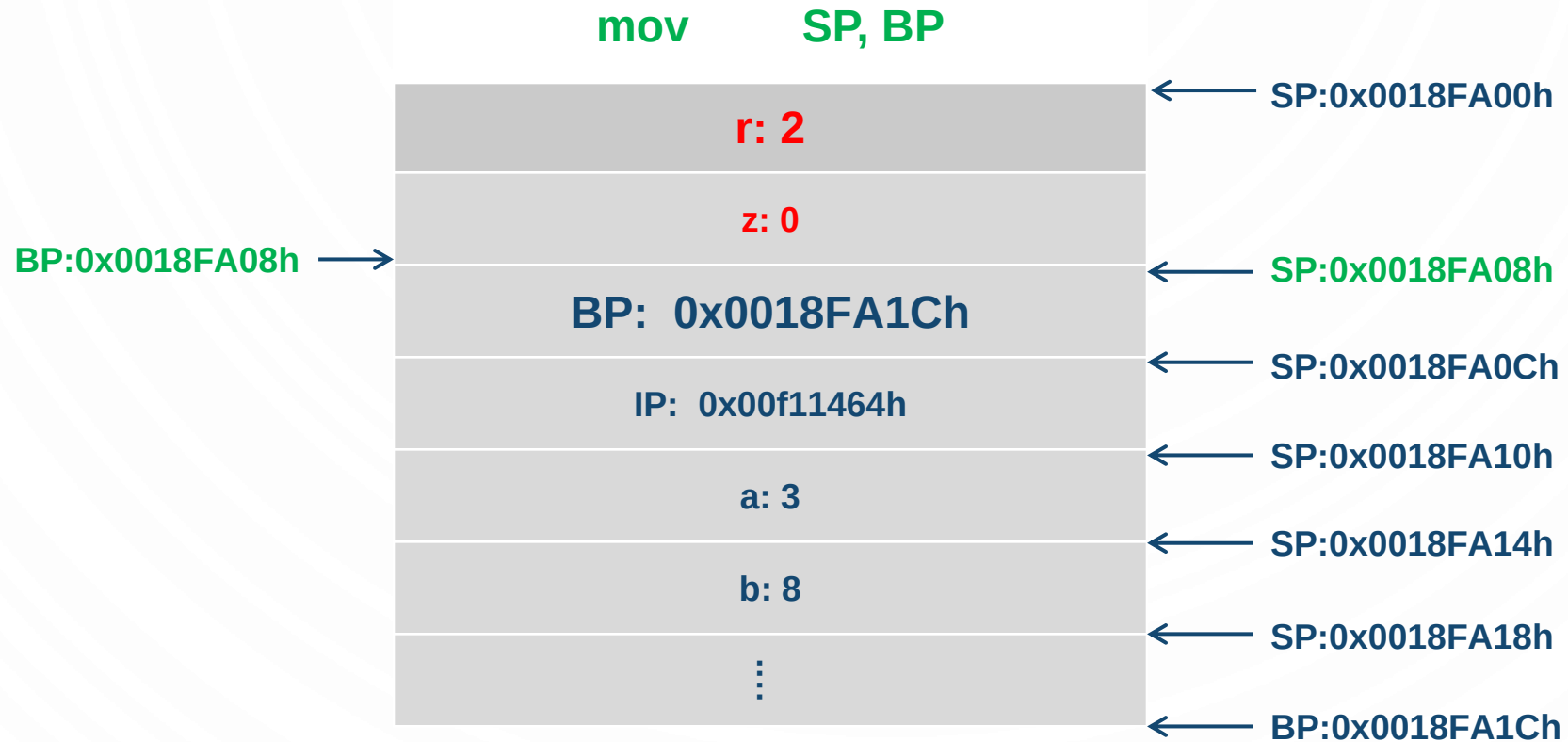
Function call simulation



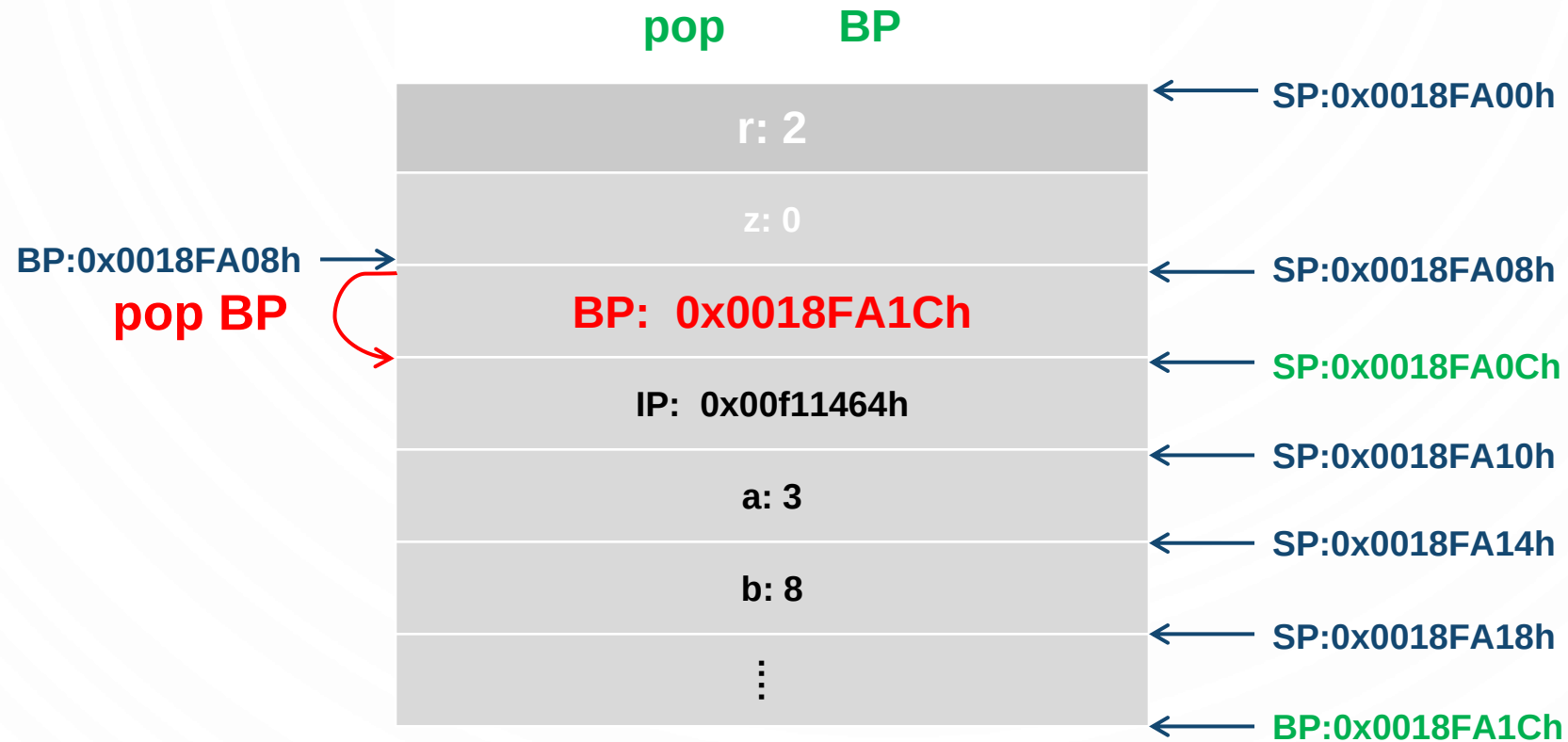
Function call simulation



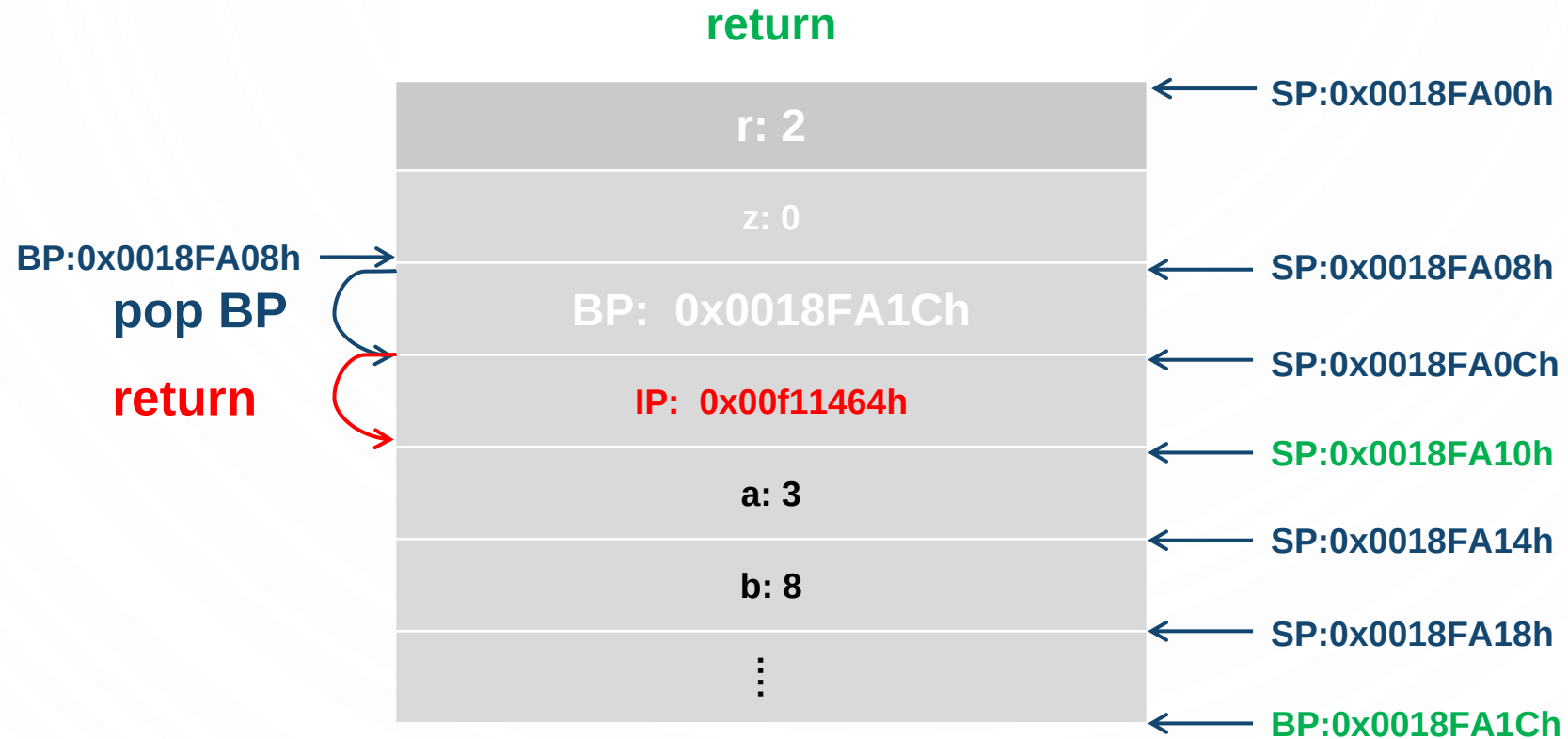
Function call simulation



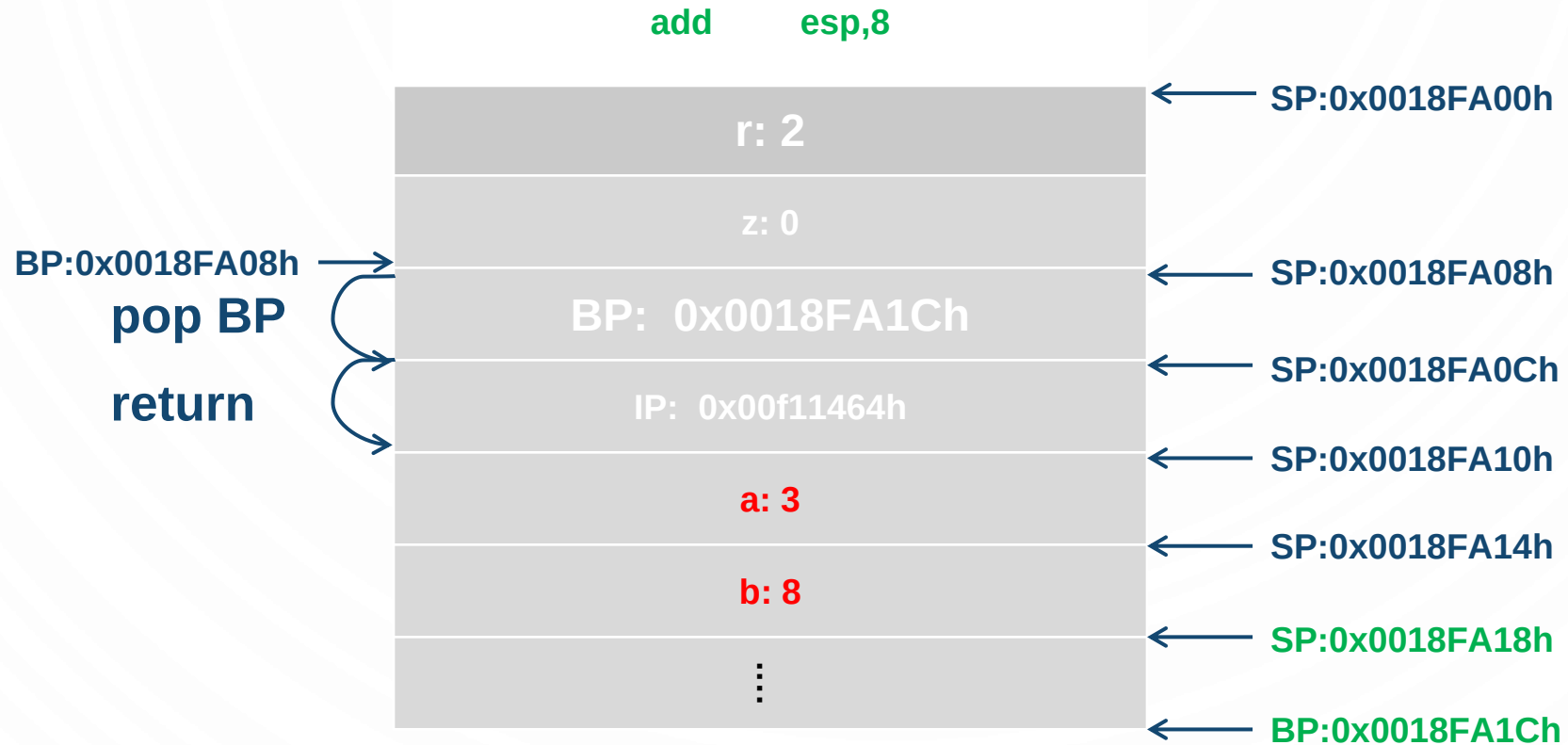
Function call simulation



Function call simulation



Function call simulation



PUSH Pseudocode

1. memory allocation (newNode);

2. newNode initialization;

3. if (stack_full)

stack overflow;

else

newNode->next = empty(head)? NULL: head;

head = newNode;



POP Pseudocode

1. declaring a temporary node (tmpNode);
2. if (stack_empty)

stack underflow;

else

value = head->info;

tmpNode = head;

head = head->next;

free(tmpNode);

return value;



Infix – Postfix – Prefix notations

- Evaluating **mathematical expressions** by using the **stack** data structure: reordering the elements so that the expression can be evaluated easily by **one single crossing** over the entire expression;
- The **Polish notation** invented by Jan Lukasiewicz also called the **prefix expression**;
- The Reverse Polish notation is called **postfix notation**.



Infix – Postfix – Prefix notations

Evaluating mathematical expressions:

- **Prefix** or Polish notation: the operators are written **before** the operands
- **Postfix** or reverse Polish notation: the operators are placed **after** the operands



Infix – Postfix – Prefix notations

The postfix or the reverse Polish notation is easily interpreted than the infix, normal mathematical expressions:

- the order of operations is straight forward;
- the parentheses are not necessary;
- the evaluation are automatically made by the use of a stack data structure;



Infix – Postfix – Prefix notations

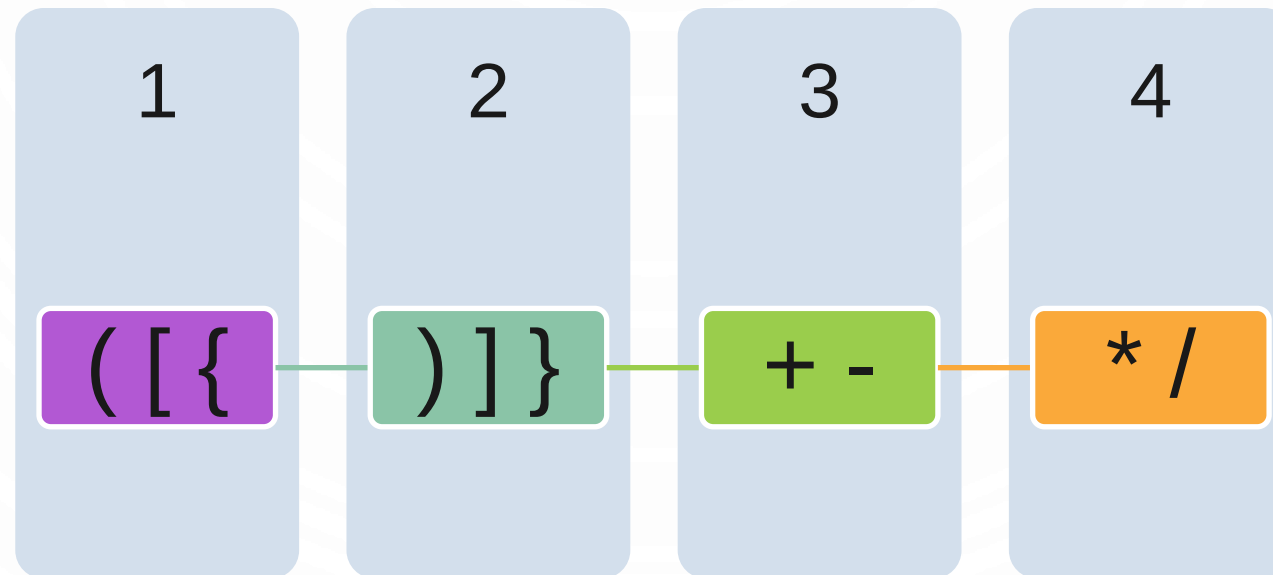
The algorithm for casting an infix expression to a postfix notation is called Dijkstra Shunting-Yard algorithm:

- using a stack for storing the operators to be transferred into the postfix notation;
- each operator has an adequate hierarchy level as depicted in the next table;



Infix – Postfix – Prefix notations

Operators hierarchy level



Infix – Postfix – Prefix notations

Mathematical expression (the infix notation)	Polish notation (prefix expression)	Reverse Polish notation (postfix expression)
$4 + 5$	$+ 4 5$	$4 5 +$
$4 + 5 * 5$	$+ 4 * 5 5$	$4 5 5 * +$
$4 * 2 + 3$	$+ * 4 2 3$	$4 2 * 3 +$
$4 + 2 + 3$	$+ + 4 2 3$	$4 2 + 3 +$
$4 * (2 + 3)$	$* 4 + 2 3$	$4 2 3 + *$



Dijkstra Shunting-Yard algorithm

- Scan the entire infix expression from left to right for tokens (operators, operands and parentheses)
- For each token, test whether:
 - If token is an operator:
 - Remove all operators at the top of the stack that have higher or equal precedence compared to the incoming operator, and add them to the postfix expression; following this step, place the new operator onto the stack;



Dijkstra Shunting-Yard algorithm

- If token is an operand, append it to the postfix expression;
- If token is a left parenthesis, push it on the stack;
- If token is a right parenthesis:
 - Remove all operators from the stack and add them to the postfix expression until a matching left parenthesis is encountered. Do not add this left parenthesis to the result, simply discard it;
- When all tokens of the infix expression have been scanned, pop all the elements from the stack and append them to the postfix expression;



Infix to Postfix

$$2^{*}(4+3)+9/3$$

Stack:

Output:2



Infix to Postfix

$$2^{*}(4+3)+9/3$$

Stack:*

Output:2



Infix to Postfix

$$2*(4+3)+9/3$$

Stack:*(

Output:2



Infix to Postfix

$$2*(4+3)+9/3$$

Stack:*(

Output:24



Infix to Postfix

$$2*(4+3)+9/3$$

Stack:*(+

Output:24



Infix to Postfix

$$2*(4+3)+9/3$$

Stack:*(+

Output:243



Infix to Postfix

$$2*(4+3)+9/3$$

Stack:*

Output:243+



Infix to Postfix

$$2*(4+3)+9/3$$

Stack: +

Output: 243+*



Infix to Postfix

$$2*(4+3)+9/3$$

Stack: +

Output: 243+*9



Infix to Postfix

$2*(4+3)+9/3$

Stack: + /

Output: 243+*9



Infix to Postfix

$$2*(4+3)+9/3$$

Stack: +/

Output: 243+*93



Infix to Postfix

$$2*(4+3)+9/3$$

Stack:

Output: 2 4 3 + * 9 3 / +



Postfix evaluation algorithm

- Scan the entire postfix expression from left to right for tokens (operands and operators):
- For each token, test whether:
 - If token is an operand is pushed on the stack;
 - If token is an operator:
 - Pop the **left** element from the stack into a **y** variable;
 - Pop the **right** element from the stack into a **x** variable;
 - The operation between **x operator y** is computed;
 - The **result** is **pushed** on the stack;
- The last value standing on the stack is the result of the expression.



Infix->Postifix->Evaluation

$$E = 5 * 3 + 2 - (3 * 2 + 1)$$



Stack VM operation

(3,3) - 1long(21) - 1int (3)		
START		END
654321		654321321
654321	DUP2_X1	65421321
65421321	POP2	654213
654213	DUP_X2	6543213
6543213	DUP_X2	65433213
65433213	POP	6543321
6543321	DUP2_X1	654321321

(3,3) - 1int(1) - 1long(32)		
START		END
654321		654321321
(4,4)		
START		END
654321		6543214321



Palindrome

Able was I ere I saw Elba



Other implementations

The *undo* mechanism in different text editors

Forward and backward feature in web browsers

Check for balanced parentheses in an expression

Expression evaluation and syntax parsing

Backtracking

Tree and Graph algorithms

Memory management for running a program



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

